

NAMA: DILLAN GUSTAV ALFARO
NPM: 2510631250030
KELAS: 2C
MATKUL: ASD

1.

```
Users > MSI THIN 15 > Documents > bubblesortint.java > bubblesortint > main(String[])
1 public class BubbleSortInt {
2
3     public static void main(String[] args) {
4
5         int[] arr = {85, 72, 93, 60, 78, 95, 88, 65, 74, 81};
6
7         int jumlahPerbandingan = 0;
8         int jumlahPertukaran = 0;
9
10        long startTime = System.nanoTime();
11
12        for (int i = 0; i < arr.length - 1; i++) {
13
14            for (int j = 0; j < arr.length - 1 - i; j++) {
15
16                jumlahPerbandingan++;
17
18                if (arr[j] < arr[j + 1]) {
19
20                    int temp = arr[j];
21                    arr[j] = arr[j + 1];
22                    arr[j + 1] = temp;
23
24                    jumlahPertukaran++;
25                }
26            }
27
28            System.out.print("Iterasi ke-" + (i + 1) + ": ");
29            for (int k = 0; k < arr.length; k++) {
30                System.out.print(arr[k] + " ");
31            }
32            System.out.println();
33        }
34
35        long endTime = System.nanoTime();
36
37        System.out.print("Hasil akhir: ");
38        for (int i = 0; i < arr.length; i++) {
39            System.out.print(arr[i] + " ");
40        }
41    }
42 }
```

Ln 7, Col 37 Spaces: 4 UTF-8 CR

```

1 public class BubbleSortInt {
3     public static void main(String[] args) {
4         if (arr[j] < arr[j + 1]) {
19
20             int temp = arr[j];
21             arr[j] = arr[j + 1];
22             arr[j + 1] = temp;
23
24             jumlahPertukaran++;
25         }
26     }
27
28     System.out.print("Iterasi ke-" + (i + 1) + ": ");
29     for (int k = 0; k < arr.length; k++) {
30         System.out.print(arr[k] + " ");
31     }
32     System.out.println();
33 }
34
35 long endTime = System.nanoTime();
36
37 System.out.print(s: "Hasil akhir: ");
38 for (int i = 0; i < arr.length; i++) {
39     System.out.print(arr[i] + " ");
40 }
41 System.out.println();
42
43 System.out.println("Jumlah perbandingan: " + jumlahPerbandingan);
44 System.out.println("Jumlah pertukaran: " + jumlahPertukaran);
45
46 double waktuMs = (endTime - startTime) / 1_000_000.0;
47 System.out.println("Waktu eksekusi: " + waktuMs + " ms");
48 }
49 }

```

```

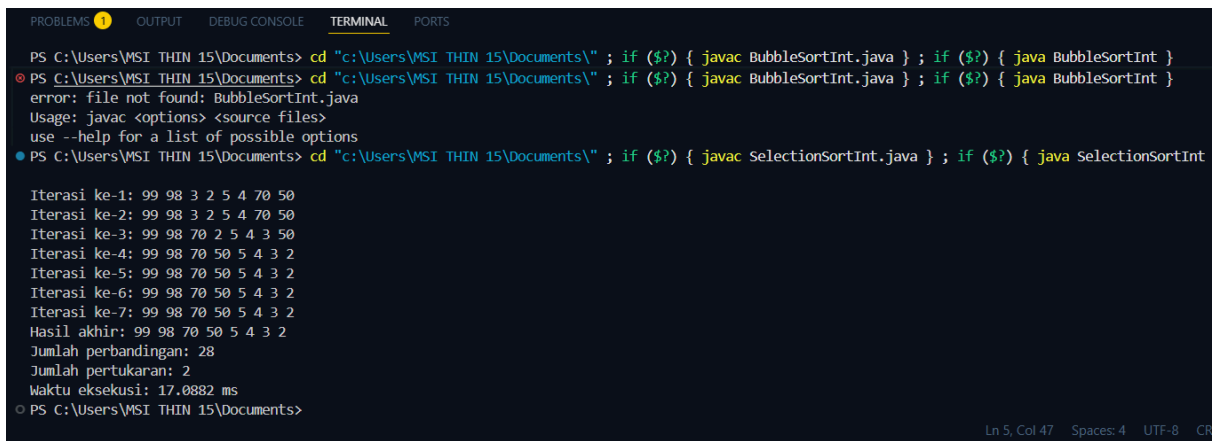
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\MSI THIN 15\Documents> cd "c:\Users\MSI THIN 15\Documents\" ; if ($?) { javac BubbleSortInt.java } ; if ($?) { java BubbleSortInt }
PS C:\Users\MSI THIN 15\Documents> cd "c:\Users\MSI THIN 15\Documents\" ; if ($?) { javac BubbleSortInt.java } ; if ($?) { java BubbleSortInt }
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index -1 out of bounds for length 10
    at BubbleSortInt.main(BubbleSortInt.java:18)
PS C:\Users\MSI THIN 15\Documents> cd "c:\Users\MSI THIN 15\Documents\" ; if ($?) { javac BubbleSortInt.java } ; if ($?) { java BubbleSortInt }
Iterasi ke-1: 85 93 72 78 95 88 65 74 81 60
Iterasi ke-2: 93 85 78 95 88 72 74 81 65 60
Iterasi ke-3: 93 85 95 88 78 74 81 72 65 60
Iterasi ke-4: 93 95 88 85 78 81 74 72 65 60
Iterasi ke-5: 95 93 88 85 81 78 74 72 65 60
Iterasi ke-6: 95 93 88 85 81 78 74 72 65 60
Iterasi ke-7: 95 93 88 85 81 78 74 72 65 60
Iterasi ke-8: 95 93 88 85 81 78 74 72 65 60
Iterasi ke-9: 95 93 88 85 81 78 74 72 65 60
Hasil akhir: 95 93 88 85 81 78 74 72 65 60
Jumlah perbandingan: 45
Jumlah pertukaran: 22
Waktu eksekusi: 26.8527 ms
PS C:\Users\MSI THIN 15\Documents>

```

Ln 7, Col 37 Spaces: 4 UTF-8

Program ini mengimplementasikan algoritma **bubble sort** untuk mengurutkan array integer secara **menurun (descending)**, yang ditandai dengan kondisi $arr[j] < arr[j + 1]$ sehingga nilai yang lebih besar dipindahkan ke kiri. Proses sorting dilakukan dengan dua perulangan bersarang: loop luar mengatur jumlah iterasi, sedangkan loop dalam membandingkan elemen yang bersebelahan dan menukarnya jika tidak sesuai urutan. Setiap perbandingan dicatat dalam variabel jumlahPerbandingan, dan setiap pertukaran nilai dicatat dalam jumlahPertukaran. Selain itu, program juga menampilkan kondisi array pada setiap iterasi untuk menunjukkan proses pengurutan secara bertahap. Setelah selesai, program menampilkan hasil akhir array yang telah terurut, jumlah total perbandingan dan pertukaran, serta waktu eksekusi yang diukur menggunakan `System.nanoTime()` dan dikonversi ke milidetik.

2. 99 98 70 2 5 4 3 50

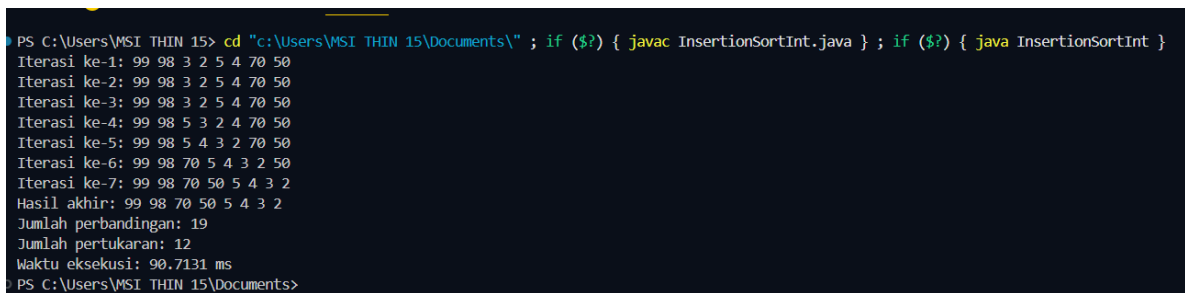


```
PS C:\Users\MSI THIN 15\Documents> cd "c:\Users\MSI THIN 15\Documents\" ; if ($?) { javac BubbleSortInt.java } ; if ($?) { java BubbleSortInt }
PS C:\Users\MSI THIN 15\Documents> cd "c:\Users\MSI THIN 15\Documents\" ; if ($?) { javac BubbleSortInt.java } ; if ($?) { java BubbleSortInt }
error: file not found: BubbleSortInt.java
Usage: javac <options> <source files>
use --help for a list of possible options
PS C:\Users\MSI THIN 15\Documents> cd "c:\Users\MSI THIN 15\Documents\" ; if ($?) { javac SelectionSortInt.java } ; if ($?) { java SelectionSortInt }

Iterasi ke-1: 99 98 3 2 5 4 70 50
Iterasi ke-2: 99 98 3 2 5 4 70 50
Iterasi ke-3: 99 98 70 2 5 4 3 50
Iterasi ke-4: 99 98 70 50 5 4 3 2
Iterasi ke-5: 99 98 70 50 5 4 3 2
Iterasi ke-6: 99 98 70 50 5 4 3 2
Iterasi ke-7: 99 98 70 50 5 4 3 2
Hasil akhir: 99 98 70 50 5 4 3 2
Jumlah perbandingan: 28
Jumlah pertukaran: 2
Waktu eksekusi: 17.0882 ms
PS C:\Users\MSI THIN 15\Documents>
```

Iterasi ke-3 seperti itu karena **selection sort selalu mencari nilai terbesar dari sisa array**, lalu menaruhnya di posisi ke-3 (indeks 2). Setelah iterasi 1 dan 2, dua nilai terbesar (99 dan 98) sudah di depan. Pada iterasi ke-3, nilai terbesar berikutnya (70) ditemukan dari sisa elemen dan ditukar ke posisi tersebut, sehingga tiga elemen pertama sudah terurut, sementara sisanya belum.

3. 99 98 3 2 5 4 70 50



```
PS C:\Users\MSI THIN 15> cd "c:\Users\MSI THIN 15\Documents\" ; if ($?) { javac InsertionSortInt.java } ; if ($?) { java InsertionSortInt }
Iterasi ke-1: 99 98 3 2 5 4 70 50
Iterasi ke-2: 99 98 3 2 5 4 70 50
Iterasi ke-3: 99 98 3 2 5 4 70 50
Iterasi ke-4: 99 98 5 3 2 4 70 50
Iterasi ke-5: 99 98 5 4 3 2 70 50
Iterasi ke-6: 99 98 70 5 4 3 2 50
Iterasi ke-7: 99 98 70 50 5 4 3 2
Hasil akhir: 99 98 70 50 5 4 3 2
Jumlah perbandingan: 19
Jumlah pertukaran: 12
Waktu eksekusi: 90.7131 ms
PS C:\Users\MSI THIN 15\Documents>
```

Iterasi ke-3 pada **insertion sort** seperti itu karena algoritma ini mengambil satu elemen (key) lalu **menyisipkannya ke posisi yang tepat di bagian kiri yang sudah terurut**. Setelah iterasi 1 dan 2, bagian kiri (indeks 0–1) sudah terurut yaitu **99 98**. Pada iterasi ke-3 (indeks 2, nilai 3), nilai tersebut dibandingkan dengan elemen di kiri (98 dan 99), tetapi karena 3 lebih kecil (descending), tidak ada pergeseran yang terjadi. Akibatnya, 3 tetap di posisinya, sehingga hasil iterasi ke-3 terlihat tidak berubah, sementara bagian kiri tetap terurut dan proses akan berlanjut ke elemen berikutnya.

4.

```
C:\Users> MS THIN 15 > Documents > InsertionSort.java > SearchingSort > main(String[])
1 import java.util.Scanner;
2 import java.util.Arrays;
3
4 public class SearchingSort {
5
6     Run | Debug
7     public static void main(String[] args) {
8
9         Scanner input = new Scanner(System.in);
10
11         int[] arr = {60, 65, 72, 75, 78, 82, 85, 88, 90, 95};
12
13         System.out.println(x: "=== Sistem Pencarian Nilai ===");
14         System.out.println("Data nilai: " + Arrays.toString(arr));
15
16         while (true) {
17             System.out.print(x: "Masukkan nilai yang ingin dicari: ");
18             int target = input.nextInt();
19
20
21             long startLinear = System.nanoTime();
22
23             int indexLinear = -1;
24             for (int i = 0; i < arr.length; i++) {
25                 if (arr[i] == target) {
26                     indexLinear = i;
27                     break;
28                 }
29             }
30
31             long endLinear = System.nanoTime();
32
33             System.out.println(x: "\n[LINEAR SEARCH]");
34             if (indexLinear != -1) {
35                 System.out.println("Data ditemukan di indeks: " + indexLinear);
36             } else {
37                 System.out.println(x: "Data tidak ditemukan.");
38             }
39             System.out.println("Waktu pencarian: " + (endLinear - startLinear) + " ns");
40
41             int[] sortedArr = arr.clone();
42             Arrays.sort(sortedArr);
43
44             long startBinary = System.nanoTime();
45
46             int left = 0, right = sortedArr.length - 1;
47             int indexBinary = -1;
```

```
public static void main(String[] args) {
    long endLinear = System.nanoTime();

    System.out.println(x: "\n[LINEAR SEARCH]");
    if (indexLinear != -1) {
        System.out.println("Data ditemukan di indeks: " + indexLinear);
    } else {
        System.out.println(x: "Data tidak ditemukan.");
    }
    System.out.println("Waktu pencarian: " + (endLinear - startLinear) + " ns");

    int[] sortedArr = arr.clone();
    Arrays.sort(sortedArr);

    long startBinary = System.nanoTime();

    int left = 0, right = sortedArr.length - 1;
    int indexBinary = -1;

    while (left <= right) {
        int mid = (left + right) / 2;

        if (sortedArr[mid] == target) {
            indexBinary = mid;
            break;
        } else if (sortedArr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    long endBinary = System.nanoTime();

    System.out.println(x: "\n[BINARY SEARCH]");
    if (indexBinary != -1) {
        System.out.println("Data ditemukan di indeks: " + indexBinary);
    } else {
        System.out.println(x: "Data tidak ditemukan.");
    }
    System.out.println("Waktu pencarian: " + (endBinary - startBinary) + " ns");

    System.out.println(x: "\n-----");
}
```

```

=== Sistem Pencarian Nilai ===
Data nilai: [60, 65, 72, 75, 78, 82, 85, 88, 90, 95]

Masukkan nilai yang ingin dicari: 75

[LINEAR SEARCH]
Data ditemukan di indeks: 3
Waktu pencarian: 800 ns

[BINARY SEARCH]
Data ditemukan di indeks: 3
Waktu pencarian: 1000 ns

-----

Masukkan nilai yang ingin dicari: █

```

Program ini merupakan aplikasi sederhana untuk mencari nilai dalam sebuah array menggunakan dua metode, yaitu **linear search** dan **binary search**. Pengguna diminta memasukkan nilai yang ingin dicari, kemudian program akan mencari nilai tersebut secara berurutan pada array menggunakan linear search dan mencatat waktu eksekusinya. Selanjutnya, array disalin dan diurutkan terlebih dahulu sebelum dilakukan binary search, karena metode ini hanya bisa digunakan pada data yang sudah terurut. Hasil pencarian dari kedua metode ditampilkan berupa indeks jika ditemukan atau pesan jika tidak ditemukan, beserta waktu pencarian dalam satuan nanodetik. Program berjalan terus-menerus dalam perulangan sehingga pengguna dapat melakukan pencarian berkali-kali.

```

5. for (int i = 0; i < 1000; i++) {
    sum += i;
}

```

Loop pada bagian ini berjalan sebanyak 1000 kali (konstan) dan tidak bergantung pada ukuran input n .

Sehingga kompleksitas waktunya adalah:

$O(1)$ (konstan)

2. Bagian 2

```

for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (arr[i] == arr[j]) {
            System.out.println("Duplikat ditemukan");
        }
    }
}

```

```
}  
}  
}
```

Pada bagian ini terdapat dua loop bersarang.

Loop luar berjalan sebanyak n kali, dan loop dalam berjalan kurang dari n kali untuk setiap iterasi.

Jumlah total perbandingan adalah:

$$n(n - 1) / 2$$

Sehingga kompleksitas waktunya adalah:

$$O(n^2)$$

Kesimpulan

Total kompleksitas dari program adalah:

$$O(1) + O(n^2) = O(n^2)$$

Karena dalam analisis Big-O hanya diambil suku yang paling dominan, maka kompleksitas akhir adalah:

$$O(n^2)$$